	Sanjivani Rural Education Society's SANJIVANI COLLEGE OF ENGINEERING **(An Autonomous Institution)** Kopargaon – 423 603, Maharashtra.	**ACAD-F-15 K**
Academic Year: 2026-27	**CIA ACTIVITY REPORT**	**Revision : 00**
Department :	**Computer Engineering**	**Date of Preparation** 15.10.2026
Course Code & Name:	**PCCO201 : Discrete Mathematics**	**Year/Sem: S.Y B. Tech/ 3rd**

Activity Title: **Social Network Friend Recommendation Using Warshall's Algorithm**

Submitted by:

Group Id- _____

Roll No	PRN No	Name of the Student	Signature
107	0125UCSM1107	Rode Dnyaneshwar	
108	0125UCSM1108	Ahemad Patwe	
109	0125UCSF1109	Alfiya Patel	
110	0125UCSM1110	Mangesh Gofane	

Abstract

Social networks are naturally modelled as directed graphs in which users are vertices and friendship (or follow) relations are directed edges. A friend recommendation system must decide which users a person is *indirectly* connected to but does not yet know. In discrete mathematics this question is answered by the **transitive closure** of a relation: the smallest transitive relation that contains the original one.

This report models a small social network as a relation on a finite set of users, represents it by a Boolean adjacency matrix, and computes its transitive closure with **Warshall's algorithm**. Correctness of the algorithm is proved formally by mathematical induction on the number of intermediate vertices allowed. The closure is then used for three tasks: (i) recommending mutual friends (friends-of-friends), (ii) detecting isolated or weakly connected users, and (iii) discovering communities of mutually reachable users.

A C++ program was implemented and run on a ten-user network. It identified three communities of size two or three, two special users (one fully isolated, one weakly connected), and produced ranked recommendations for every user. A benchmark on random graphs of 50 to 800 users confirmed the $O(n^3)$ running time: doubling n multiplied the time by about seven to eight. Limitations and extensions such as bit-set optimisation and strongly-connected-component algorithms are discussed.

Index Terms — Discrete mathematics, relations, transitive closure, Warshall's algorithm, directed graph, friend recommendation, social network, C++.

Index

- Abstract
- Chapter 1 Introduction
- Chapter 2 Literature Survey
- Chapter 3 System Design
- Chapter 4 Implementation Details
- Chapter 5 Result Screenshots and Analysis
- Chapter 6 Conclusion
- References
- Annexure 1: GitHub Upload Screenshot
- Annexure 2: Presentation Geo-Tag Photo
- Annexure 3: Certificate of Online Course

Chapter 1 Introduction

1.1 Background

Discrete mathematics studies countable structures such as sets, relations, graphs and logical statements. Two of its central ideas are the **relation** and the **directed graph**, and they are really the same object seen in two ways: a relation R on a set V is a set of ordered pairs, and the same pairs can be drawn as arrows between vertices [1], [2].

Online social networks such as Instagram, LinkedIn and Facebook are huge directed graphs. Each user is a vertex, and an edge from u to v means "u is connected to v" (u follows v, or u has added v as a friend). The network grows by recommending new connections, and one of the most effective signals for such a recommendation is *indirect connectivity*: if Aarav knows Bhavna and Bhavna knows Chinmay, then Chinmay is a natural suggestion for Aarav. Following such chains of any length is exactly the idea of the transitive closure of a relation.

Warshall's algorithm, published in 1962, computes the transitive closure of a relation in three nested loops over a Boolean matrix [3]. It is short, easy to prove correct by induction, and fits naturally into a first course on discrete mathematics, which makes it an ideal vehicle for connecting theory with a realistic application.

1.2 Problem Statement

The project represents users as nodes and their friendships as directed edges. Using Warshall's algorithm, the system calculates the **transitive closure** to find indirect connections, and uses it to recommend:

- mutual friends (friends-of-friends);
- isolated users with fewer connections; and
- communities or clusters of users in the network.

1.3 Objectives

- To study relations, directed graphs, Boolean matrices and the transitive closure.
- To prove the correctness of Warshall's algorithm by mathematical induction.
- To model a social network as a directed graph and compute its closure in C++.
- To derive friend recommendations, isolated users and communities from the closure matrix.
- To analyse the time and space complexity of the method and verify it experimentally.
- To connect a concept of discrete mathematics with a practical computing application.

1.4 Scope and Limitations

The report covers a static network with a fixed number of users, stored as an adjacency matrix. Edges are unweighted and directed, and no user attributes (age, location, interests) are used. The demonstration network has ten users so that every matrix can be shown and checked by hand; the benchmark uses random graphs of up to 800 users. Large real networks with millions of users would require sparse data structures and approximate methods, which are outside the scope of this report. Timing results depend on the machine and compiler and show trends rather than absolute values.

1.5 Methodology

The work followed four stages. First, the theory of relations, closures and graph reachability was collected from standard textbooks [1], [2], [6]. Second, Warshall's algorithm was studied and its correctness proved by induction. Third, a friend-recommendation model was designed on top of the closure matrix. Fourth, the model was coded in C++, run on a sample network, and benchmarked. Every group member contributed to one or more stages, and the results are consolidated in this report and in the accompanying presentation.

1.6 Organization of the Report

Chapter 2 is the literature survey on relations, closures, Warshall's algorithm and recommendation. Chapter 3 gives the system design, proof and algorithms. Chapter 4 documents the C++ implementation. Chapter 5 presents results, performance analysis and solved problems. Chapter 6 concludes the report and suggests future work.

Chapter 2 Literature Survey

2.1 Relations and Their Properties

A binary relation R on a set V is a subset of V × V. We write u R v when (u, v) ∈ R. The relation is **reflexive** if every u satisfies u R u, **symmetric** if u R v implies v R u, and **transitive** if u R v and v R w together imply u R w [1], [7].

A friendship relation shows why transitivity matters. “Follows” on a social network is usually *not* transitive: Aarav following Bhavna and Bhavna following Chinmay does not mean that Aarav follows Chinmay. The gap between the relation as it is and the relation as it would be if it were transitive is precisely the set of friend suggestions.

2.2 Directed Graphs and Boolean Matrices

A directed graph G = (V, E) has a set of vertices V and a set of ordered pairs E ⊆ V × V called edges. For n users numbered 1 to n, the **adjacency matrix** A is the n × n Boolean matrix with A[i][j] = 1 if and only if there is an edge from user i to user j. A **path** of length m from u to v is a sequence of vertices u = x_0, x_1, …, x_m = v in which every consecutive pair is an edge. The vertices x_1, …, x_{m−1} are the **intermediate vertices** of the path [1], [6].

Boolean matrices are multiplied with AND in place of multiplication and OR in place of addition. The Boolean power A^[m] has a 1 in position (i, j) exactly when a path of length m joins i to j. This is the matrix form of the statement that composing a relation with itself m times describes chains of m steps.

2.3 Transitive Closure

The **transitive closure** R^+ of a relation R is the smallest transitive relation containing R. Equivalently, u R^+ v exactly when there is a path of length at least one from u to v in the graph of R. In matrix form,

$$\mathbf{A^+ = A \vee A^{[2]} \vee A^{[3]} \vee \dots \vee A^{[n]}} \quad (1)$$

because a shortest path between two vertices of an n-vertex graph never needs more than n edges [1]. The closure is also called the **connectivity relation**. Computing (1) directly by repeated Boolean multiplication costs $O(n^4)$ operations, which motivates a faster method.

2.4 Warshall’s Algorithm

Stephen Warshall published his algorithm in 1962 in a short paper titled *A theorem on Boolean matrices* [3]. Bernard Roy had described essentially the same procedure in 1959 [4], and the same triple loop with additions in place of Boolean operations gives Floyd’s all-pairs shortest-path algorithm [5], which is why the family is often called the Floyd–Warshall algorithm [8].

The key idea is to allow the intermediate vertices of a path to be introduced one at a time. Let the vertices be numbered 1 to n. After the k-th stage, the matrix records every pair that is joined by a path whose intermediate vertices all lie in {1, …, k}. When k reaches n, every path is allowed, so the matrix is the transitive closure. The algorithm needs only n^2 bits of storage and runs in $\Theta(n^3)$ time [8].

2.5 Reachability, Strong Connectivity and Communities

Two vertices u and v are **mutually reachable** when u R^+ v and v R^+ u. Mutual reachability is an equivalence relation on the set of vertices that lie on a cycle (and each isolated vertex forms a class on its own), and its classes are the **strongly connected components** of the graph [1], [8]. In a social network, a strongly connected component is a group of users who can all reach one another through chains of connections, which is a sensible mathematical definition of a small community. Linear-time algorithms such as Tarjan's [9] find these components for very large graphs, whereas the closure-based method used here is simpler and ideal for teaching.

2.6 Link Prediction and Friend Recommendation

Predicting which new edges will appear in a social network is known as the **link-prediction problem**. Liben-Nowell and Kleinberg showed that simple measures based on the existing graph structure, such as the number of common neighbours, already predict future links much better than chance [10]. Common-neighbour counting is the same idea as "mutual friends": two users with many mutual friends are likely to become friends. Network science textbooks describe this triadic closure effect as one of the most reliable features of real social networks [11]. The transitive closure generalises it from two-step to chains of any length.

2.7 Applications of Transitive Closure

- **Social networks:** friend suggestions, influence reach, community detection.
- **Databases:** recursive queries such as "all employees below this manager" are closure computations.
- **Compilers and software engineering:** finding every function that can eventually call another, or every module that depends on another.
- **Networks and transport:** deciding whether a route of any number of hops exists between two stations.
- **Epidemiology and information spread:** who can eventually be reached from an initial set of people.

Chapter 3 System Design

3.1 Overall Design

The system is a small C++ console program. It stores the network as an adjacency matrix, runs Warshall's algorithm to obtain the closure, and then performs three analyses on the closure. The flow is:

Network (users + edges) → Adjacency matrix A → Warshall's algorithm → Closure matrix A^+ → { Recommendations, Isolated users, Communities } → Output.

The design is deliberately simple: each task is a single function and the program only orchestrates input, computation and printing, so that attention stays on the mathematics.

3.2 Mathematical Model

Let V = {v_0, v_1, …, v_{n−1}} be the set of users and let E ⊆ V × V be the set of directed friendship edges. Define the relation F by u F v if (u, v) ∈ E. The adjacency matrix is A[i][j] = 1 when v_i F v_j and 0 otherwise. The closure matrix C = A^+ is defined by C[i][j] = 1 when there is a path of length at least one from v_i to v_j.

The three outputs of the system are defined from A and C:

- **Recommendations for user i:** the set Rec(i) = { j : j ≠ i, C[i][j] = 1 and A[i][j] = 0 }. These are users that i can reach indirectly but is not yet connected to.
- **Mutual-friend score:** m(i, j) = |{ k : A[i][k] = 1 and A[k][j] = 1 }|, the number of two-step paths from i to j. A larger score means a stronger recommendation.
- **Isolated or weakly connected users:** user i is *isolated* if it has no edges at all (in-degree plus out-degree is 0, and C[i][j] = 0 for every j ≠ i). It is *weakly connected* when the total degree is at most one.
- **Communities:** users i and j belong to the same community when C[i][j] = 1 and C[j][i] = 1 (mutual reachability). A user who lies on no cycle forms a community of one.

3.3 Warshall's Algorithm

```
Algorithm WARSHALL(A, n)
Input  : n × n Boolean adjacency matrix A
Output : transitive closure matrix R = A+
1. R <- A
2. for k <- 0 to n-1 do          // allow vertex k as intermediate
3.     for i <- 0 to n-1 do
4.         for j <- 0 to n-1 do
5.             R[i][j] <- R[i][j] OR ( R[i][k] AND R[k][j] )
6. return R
```

The rule on line 5 says: user i reaches user j if i already reached j, or if i reaches k and k reaches j. The outer loop over k is the essential one; interchanging the loops gives a wrong answer.

3.4 Proof of Correctness by Induction

Let R_k denote the matrix after the outer loop has processed vertices 0, …, k−1 (so R_0 = A and R_n is the final matrix).

Claim P(k). For $0 \le k \le n$, R_k[i][j] = 1 if and only if there is a path of length at least one from v_i to v_j all of whose intermediate vertices belong to $\{v_0, \ldots, v_{k-1}\}$.

Basis step (k = 0). No intermediate vertex is allowed, so the only such paths are single edges. R_0 = A, so R_0[i][j] = 1 exactly when $(v_i, v_j) \in E$. Hence P(0) is true.

Inductive hypothesis. Assume P(k) holds for some k with $0 \le k < n$.

Inductive step. Consider stage k, which sets R_{k+1}[i][j] = R_k[i][j] OR (R_k[i][k] AND R_k[k][j]). We show P(k+1).

- (⇐) Suppose a path from v_i to v_j has all intermediate vertices in $\{v_0, \ldots, v_k\}$. If v_k is not an intermediate vertex, the path qualifies for R_k, so R_k[i][j] = 1 by the hypothesis. If v_k is an intermediate vertex, we may assume it occurs exactly once (a repeated visit forms a cycle that can be cut out without destroying the path). The path then splits into a part from v_i to v_k and a part from v_k to v_j, each with intermediate vertices in $\{v_0, \ldots, v_{k-1}\}$. By the hypothesis R_k[i][k] = 1 and R_k[k][j] = 1, so the stage-k rule sets R_{k+1}[i][j] = 1.
- (⇒) Conversely, if R_{k+1}[i][j] = 1 then either R_k[i][j] = 1, giving a path with intermediates in $\{v_0, \ldots, v_{k-1}\}$, or both R_k[i][k] and R_k[k][j] equal 1. In the second case, joining the two paths gives a walk from v_i to v_j with intermediates in $\{v_0, \ldots, v_k\}$, and a walk contains a path between the same endpoints with no more intermediate vertices.

Thus P(k) → P(k+1). **Conclusion.** By the principle of mathematical induction P(k) holds for all k from 0 to n. For k = n every vertex is allowed as an intermediate vertex, so R_n[i][j] = 1 exactly when v_i reaches v_j, which means $R_n = A^+$. ■

Remark (in-place update). The algorithm overwrites R while it runs. This is safe because during stage k the entries R[i][k] and R[k][j] do not change: R[i][k] becomes R[i][k] OR (R[i][k] AND R[k][k]), which equals R[i][k]. The same holds for row k.

3.5 Algorithm for Friend Recommendation

```
Algorithm RECOMMEND(A, C, n)
for i <- 0 to n-1 do
    for j <- 0 to n-1 do
        if i != j and C[i][j] = 1 and A[i][j] = 0 then
            m <- number of k with A[i][k] = 1 and A[k][j] = 1
            suggest j to i  (with mutual-friend score m)
```

3.6 Algorithm for Isolated Users and Communities

```
Algorithm ISOLATED(A, C, n)
for i <- 0 to n-1 do
    out <- sum of A[i][j] over j ;  in <- sum of A[j][i] over j
    reach <- number of j != i with C[i][j] = 1
    if out + in <= 1 or reach = 0 then report i

Algorithm COMMUNITIES(C, n)
mark all users unvisited
for i <- 0 to n-1 do
    if i unvisited then
        community <- { j : j = i or (C[i][j] = 1 and C[j][i] = 1) }
        mark all members visited ; report community
```

Because mutual reachability is an equivalence relation on users who lie on cycles, the communities reported by COMMUNITIES do not overlap.

3.7 Complexity Analysis

Task	Time	Extra space	Remark
Warshall closure	$\Theta(n^3)$	$O(n^2)$	three nested loops of n iterations
Recommendations	$O(n^3)$	$O(1)$	naive mutual-friend counting for every pair
Isolated users	$O(n^2)$	$O(1)$	one scan of the matrix
Communities	$O(n^2)$	$O(n)$	one scan of the closure

The closure dominates the cost: the innermost statement runs $n \times n \times n = n^3$ times, so the total is $\Theta(n^3)$. The matrix needs n^2 entries, so the memory is $\Theta(n^2)$. Doubling the number of users should therefore multiply the running time by about eight.

3.8 Test Plan

The test plan combines small networks whose closure can be written down by hand with the demonstration network of Chapter 5.

Test	Network	Expected closure
T1	No edges, n = 3	All zeros; every user isolated
T2	Single edge A → B	Only the entry (A, B) is 1
T3	Chain A → B → C → D	$n(n-1)/2 = 6$ ones above the diagonal; no community of size > 1
T4	Cycle A → B → C → D → A	All 16 entries are 1; one community of four users
T5	Two disjoint cycles	Two blocks of ones; two communities; no entries between them
T6	Ten-user demonstration network	See Chapter 5

Chapter 4 Implementation Details

4.1 Development Environment

Item	Details
Programming language	C++ (ISO C++ standard [12])
Compiler	GNU g++
Operating system	Ubuntu Linux (x86-64)
Optimisation level	-O0 (no optimisation), so that loops are not rewritten
Data structure	Two-dimensional int arrays for A and the closure R
Version control	Git / GitHub (see Annexure 1)

4.2 Main Program

Listing 1 gives the complete program. It stores a ten-user network, runs Warshall's algorithm and prints the closure, the recommendations, the isolated users and the communities.

Listing 1. friend.cpp — friend recommendation using transitive closure

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

const int N = 10;
string name[N] = {"Aarav","Bhavna","Chinmay","Disha","Esha",
                  "Farhan","Gauri","Harsh","Isha","Jay"};
int adj[N][N], R[N][N];

void addEdge(int u, int v) { adj[u][v] = 1; }

void warshall() {
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++) R[i][j] = adj[i][j];
    for (int k = 0; k < N; k++)
        for (int i = 0; i < N; i++)
            for (int j = 0; j < N; j++)
                R[i][j] = R[i][j] || (R[i][k] && R[k][j]);
}

void printMat(int M[N][N], const char* title) {
    cout << title << "\n    ";
    for (int j = 0; j < N; j++) cout << name[j][0] << " ";
    cout << "\n";
    for (int i = 0; i < N; i++) {
        cout << name[i][0] << " | ";
        for (int j = 0; j < N; j++) cout << M[i][j] << " ";
        cout << "\n";
    }
}

int main() {
    // directed friendship edges: u follows v
    addEdge(0,1); addEdge(1,2); addEdge(2,0); addEdge(2,3);
    addEdge(3,4); addEdge(4,5); addEdge(5,3); addEdge(6,7);
    addEdge(7,6); addEdge(7,5); addEdge(9,0);

    printMat(adj, "Adjacency matrix A:");
    warshall();
    printMat(R, "\nTransitive closure A+ (Warshall):");

    cout << "\n--- Friend recommendations ---\n";
```

```
    for (int i = 0; i < N; i++) {
        cout << name[i] << " -> ";
        bool any = false;
        for (int j = 0; j < N; j++)
            if (i != j && R[i][j] && !adj[i][j]) {
                int mutual = 0;
                for (int k = 0; k < N; k++)
                    if (adj[i][k] && adj[k][j]) mutual++;
                cout << name[j];
                if (mutual) cout << "(mutual:" << mutual << ")";
                cout << " ";
                any = true;
            }
        if (!any) cout << "none";
        cout << "\n";
    }

    cout << "\n--- Isolated / weakly connected users ---\n";
    for (int i = 0; i < N; i++) {
        int out = 0, in = 0, reach = 0;
        for (int j = 0; j < N; j++) {
            out += adj[i][j]; in += adj[j][i];
            if (i != j && R[i][j]) reach++;
        }
        if (out + in <= 1 || reach == 0)
            cout << name[i] << ": out=" << out << ", in=" << in
                 << ", reachable=" << reach << "\n";
    }

    cout << "\n--- Communities (mutually reachable users) ---\n";
    vector<bool> seen(N, false);
    int c = 0;
    for (int i = 0; i < N; i++) if (!seen[i]) {
        cout << "Community " << ++c << ": ";
        for (int j = 0; j < N; j++)
            if (i == j || (R[i][j] && R[j][i])) {
                seen[j] = true; cout << name[j] << " ";
            }
        cout << "\n";
    }
    return 0;
}
```

4.3 Explanation of the Code

- **adj and R.** adj stores the original network and R receives the closure, so that the original friendships are still available for recommendations.
- **addEdge(u, v).** Records that user u is connected to user v. The relation is directed, so addEdge(0,1) does not create the edge from 1 to 0.
- **warshall().** Copies adj into R and then applies the rule R[i][j] = R[i][j] OR (R[i][k] AND R[k][j]) inside three nested loops, with k as the outermost loop exactly as in the proof of Section 3.4.
- **Recommendation block.** A pair (i, j) is suggested when R[i][j] is 1 but adj[i][j] is 0. The inner loop over k counts two-step paths, which is the mutual-friend score.
- **Isolated block.** Computes the out-degree, in-degree and the number of reachable users, and reports users with total degree at most one or with nothing reachable.
- **Community block.** Groups users i and j when each can reach the other. The array seen prevents a community from being printed twice.

4.4 Benchmark Program

To check the O(n³) behaviour, a second program generates a random directed graph for each size n, in which each possible edge appears with probability 3 percent, and measures the time taken by the Warshall triple loop with the high-resolution clock of the C++ chrono library. A fixed random seed was used so that the experiment can be repeated.

Listing 2. bench.cpp — timing of Warshall's algorithm (excerpt)

```
srand(1);
for (int n : {50, 100, 200, 400, 800}) {
    vector<vector<char>> R(n, vector<char>(n, 0));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            if (i != j && rand() % 100 < 3) R[i][j] = 1;
    auto a = chrono::high_resolution_clock::now();
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                R[i][j] = R[i][j] || (R[i][k] && R[k][j]);
    auto b = chrono::high_resolution_clock::now();
    // print n, number of edges, and duration in milliseconds
}
```

4.5 Implementation Notes

- Compile with g++ -O0 -o friend friend.cpp and run with ./friend. Optimisation was disabled for the benchmark so that the compiler does not restructure the loops being studied.
- User names are stored in an array and users are identified by their index, which keeps the matrix code simple.
- A production version would read the network from a file or database, validate indices and use sparse structures, because an n × n matrix is not practical for millions of users.
- The source code was uploaded to a GitHub repository by the group (screenshot in Annexure 1), and each member contributed to coding, testing or documentation.

Chapter 5 Result Screenshots and Analysis

5.1 Demonstration Network

The demonstration network has ten users and eleven directed edges. An edge u → v means that u is connected to (follows) v.

From	To
Aarav	Bhavna
Bhavna	Chinmay
Chinmay	Aarav, Disha
Disha	Esha
Esha	Farhan
Farhan	Disha
Gauri	Harsh
Harsh	Gauri, Farhan
Isha	(none)
Jay	Aarav

The network was designed to contain every situation of interest: a cycle of three users (Aarav, Bhavna, Chinmay), a second cycle (Disha, Esha, Farhan) reached from the first by a one-way edge, a mutual pair (Gauri, Harsh) that points into the second cycle, one user with no connections (Isha) and one user with a single outgoing connection (Jay).

5.2 Program Output

The following is the actual console output of Listing 1. Replace these blocks with your own terminal screenshots (Fig. 1 to Fig. 4) when you submit.

Fig. 1. Adjacency matrix A of the network

```
Adjacency matrix A:
    A B C D E F G H I J
A | 0 1 0 0 0 0 0 0 0 0
B | 0 0 1 0 0 0 0 0 0 0
C | 1 0 0 1 0 0 0 0 0 0
D | 0 0 0 0 1 0 0 0 0 0
E | 0 0 0 0 0 1 0 0 0 0
F | 0 0 0 1 0 0 0 0 0 0
G | 0 0 0 0 0 0 0 1 0 0
H | 0 0 0 0 0 1 1 0 0 0
I | 0 0 0 0 0 0 0 0 0 0
J | 1 0 0 0 0 0 0 0 0 0
```

Fig. 2. Transitive closure matrix computed by Warshall’s algorithm

```
Transitive closure A+ (Warshall):
    A B C D E F G H I J
A | 1 1 1 1 1 1 0 0 0 0
B | 1 1 1 1 1 1 0 0 0 0
C | 1 1 1 1 1 1 0 0 0 0
D | 0 0 0 1 1 1 0 0 0 0
E | 0 0 0 1 1 1 0 0 0 0
```

```
F | 0 0 0 1 1 1 0 0 0 0
G | 0 0 0 1 1 1 1 1 0 0
H | 0 0 0 1 1 1 1 1 0 0
I | 0 0 0 0 0 0 0 0 0 0
J | 1 1 1 1 1 1 0 0 0 0
```

Fig. 3. Friend recommendations and isolated users

```
--- Friend recommendations (indirect, not yet friends) ---
Aarav -> Chinmay(mutual:1) Disha Esha Farhan
Bhavna -> Aarav(mutual:1) Disha(mutual:1) Esha Farhan
Chinmay -> Bhavna(mutual:1) Esha(mutual:1) Farhan
Disha -> Farhan(mutual:1)
Esha -> Disha(mutual:1)
Farhan -> Esha(mutual:1)
Gauri -> Disha Esha Farhan(mutual:1)
Harsh -> Disha(mutual:1) Esha
Isha -> none
Jay -> Bhavna(mutual:1) Chinmay Disha Esha Farhan

--- Isolated / weakly connected users ---
Isha: out=0, in=0, reachable=0
Jay: out=1, in=0, reachable=6
```

Fig. 4. Communities

```
--- Communities (mutually reachable users) ---
Community 1: Aarav Bhavna Chinmay
Community 2: Disha Esha Farhan
Community 3: Gauri Harsh
Community 4: Isha
Community 5: Jay
```

5.3 Analysis of the Closure

Reading the closure matrix row by row gives the whole analysis.

- **Rows A, B and C are identical** (reaching A to F), and so are rows D, E and F (reaching D to F). Users inside a cycle can all reach one another, which is why each cycle forms one community.
- **Row A has no entry in columns D to F in the original matrix**, yet the closure has ones there. These new ones, for example (Aarav, Disha), are the chains Aarav → Bhavna → Chinmay → Disha that no single friendship exposes.
- **Rows D, E, F have zeros in columns A to C.** The edge Chinmay → Disha is one-way, so Disha, Esha and Farhan cannot reach the first community. The closure respects the direction of the edges.
- **Row I is entirely zero** and column I is entirely zero: Isha neither reaches nor is reached by anyone.
- **Row J reaches A to F but column J is zero.** Jay follows Aarav, so Jay reaches everyone Aarav reaches, but nobody reaches Jay.

5.4 Analysis of the Recommendations

- **Mutual friends.** Aarav is recommended Chinmay with a mutual-friend score of 1 because Aarav → Bhavna → Chinmay. Disha is recommended Farhan (via Esha) and Esha is recommended Disha (via Farhan), exactly as expected in a cycle.
- **Longer chains.** Aarav is also recommended Disha, Esha and Farhan with no mutual friend at all. These are reachable only through chains of three or more edges, and they are weaker suggestions than the ones with a positive mutual-friend score. This is why the program prints the score.
- **Isolated users.** Isha gets no recommendation because nobody is reachable. A real system would suggest popular users or contacts imported from the phone book to such a user (the *cold-start* problem).
- **Weakly connected users.** Jay has a single connection but is recommended five users, because everything Aarav can reach is reachable by Jay. In practice only the top few suggestions by score would be shown.
- **Communities.** The program reports five groups: {Aarav, Bhavna, Chinmay}, {Disha, Esha, Farhan}, {Gauri, Harsh}, {Isha} and {Jay}. Gauri and Harsh form a separate community even though Harsh follows Farhan, because nobody in the second cycle can reach back.

5.5 Performance Comparison

Table 1 and the benchmark of Listing 2 give the measured running time of Warshall's algorithm on random graphs. The last column is the ratio of each time to the previous one when n is doubled.

Table 1. Running time of Warshall's algorithm on random graphs (-O0)

n (users)	Edges	Time (ms)	Ratio to previous row
50	71	0.80	—
100	307	6.12	7.7
200	1157	42.06	6.9
400	4854	289.62	6.9
800	19150	2167.02	7.5

Since the running time is proportional to n^3, doubling n should multiply it by $2^3 = 8$. The measured ratios of 6.9 to 7.7 are close to this value; the small shortfall comes from cache effects and from the data-dependent short-circuit evaluation of the AND operator. The measurements confirm the $\Theta(n^3)$ complexity derived in Section 3.7. At 800 users the computation already takes about two seconds, which shows why very large networks cannot use this method directly.

Absolute times depend on the machine and compiler, but the growth trend is a property of the algorithm.

5.6 Problem Solutions

The following problems illustrate the technique.

Problem 1. Find the transitive closure of R = {(1,2), (2,3)} on {1, 2, 3}. *Solution:* Stage k = 2 sets R[1][3] = R[1][2] AND R[2][3] = 1. No other entry changes, so R^+ = {(1,2), (2,3), (1,3)}.

Problem 2. Find the transitive closure of R = {(a,b), (b,c), (c,a)}. *Solution:* The graph is a directed cycle. Stage a adds (c,b); stage b adds (a,c), (c,c); stage c adds (b,a), (a,a), (b,b). All nine pairs are present, so $R^+ = V \times V$ and the three users form one community.

Problem 3. How many operations does Warshall's algorithm perform for n = 1000 users? *Solution:* The innermost statement runs $n^3 = 1000^3 = 10^9$ times.

Problem 4. Prove by induction that the transitive closure of the chain $v_1 \to v_2 \to \dots \to v_n$ contains exactly n(n−1)/2 pairs. *Solution:* The closure contains the pair (v_i, v_j) exactly when i < j. *Basis:* for n = 1 there are no edges and no pairs, and 1·0/2 = 0. *Step:* assume the claim for a chain of k users. Appending v_{k+1} with the edge v_k → v_{k+1} adds exactly the k pairs (v_i, v_{k+1}) for i = 1, …, k. The total becomes k(k−1)/2 + k = k(k+1)/2, which is the formula for k + 1. ■

Problem 5. Can a user appear in two different communities? *Solution:* No. Mutual reachability is reflexive, symmetric and transitive on the users it applies to, so it partitions them into disjoint classes.

Chapter 6 Conclusion

6.1 Conclusion

This project modelled a social network as a relation on a set of users and showed that its transitive closure, computed by Warshall's algorithm, gives a precise mathematical basis for friend recommendation. The correctness of the algorithm was proved by mathematical induction on the set of vertices allowed as intermediate vertices.

A C++ program computed the closure of a ten-user network and derived three results from it: indirect connections to recommend (ranked by the number of mutual friends), users who are isolated or weakly connected, and communities defined by mutual reachability. The communities found were {Aarav, Bhavna, Chinmay}, {Disha, Esha, Farhan}, {Gauri, Harsh}, {Isha} and {Jay}, which agree with the hand analysis of the closure matrix. Benchmarks on random graphs of 50 to 800 users showed that doubling the number of users multiplies the running time by about seven, in agreement with the $\Theta(n^3)$ analysis.

The project shows that abstract concepts of discrete mathematics, namely relations, Boolean matrices, paths and induction, are the working parts of a practical application.

6.2 Future Scope

- Replace the full recommendation list by a ranked top-k list using the number of mutual friends, path length or both.
- Store each matrix row as a bit-set so that one machine word processes many columns at once, which speeds up Warshall's algorithm by a large constant factor.
- Use Tarjan's linear-time algorithm [9] for strongly connected components on very large sparse networks.
- Add weights (for example interaction frequency) and use the Floyd–Warshall shortest-path variant [5] to rank suggestions by closeness.
- Handle a dynamic network in which friendships are added and removed, by updating the closure incrementally instead of recomputing it.
- Add a file-based input and a menu-driven interface, and visualise the network and its communities graphically.

References

[1] K. H. Rosen, *Discrete Mathematics and Its Applications*, 8th ed. New York, NY, USA: McGraw-Hill Education, 2019.

[2] C. L. Liu, *Elements of Discrete Mathematics*, 2nd ed. New York, NY, USA: McGraw-Hill, 1985.

[3] S. Warshall, "A theorem on Boolean matrices," *Journal of the ACM*, vol. 9, no. 1, pp. 11–12, 1962.

[4] B. Roy, "Transitivité et connexité," *Comptes Rendus de l'Académie des Sciences*, vol. 249, pp. 216–218, 1959.

[5] R. W. Floyd, "Algorithm 97: Shortest path," *Communications of the ACM*, vol. 5, no. 6, p. 345, 1962.

[6] R. Johnsonbaugh, *Discrete Mathematics*, 8th ed. Boston, MA, USA: Pearson, 2018.

[7] S. S. Epp, *Discrete Mathematics with Applications*, 5th ed. Boston, MA, USA: Cengage Learning, 2019.

[8] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[9] R. E. Tarjan, "Depth-first search and linear graph algorithms," *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, 1972.

[10] D. Liben-Nowell and J. Kleinberg, "The link-prediction problem for social networks," *Journal of the American Society for Information Science and Technology*, vol. 58, no. 7, pp. 1019–1031, 2007.

[11] M. E. J. Newman, *Networks: An Introduction*. Oxford, U.K.: Oxford University Press, 2010.

[12] *Programming Languages — C++*, ISO/IEC Standard 14882:2020, International Organization for Standardization, Geneva, Switzerland, 2020.

Annexure 1

GitHub Upload Screenshot

[Paste the screenshot of your GitHub repository upload here]

Screenshot of the project repository on GitHub showing friend.cpp and the report.

Annexure 2

Presentation Geo-Tag Photo

[Paste the geo-tagged presentation photo here]

Geo-tagged photograph taken during the group PowerPoint presentation.

Annexure 3

Certificate of Online Course Related to Discrete Mathematics

[Paste the online course certificate here]